Judgment Detection

Course: Social Media Mining CSE472
Instructor: Prof Huan Liu
Semester: Fall 2025

Team Members:

- Sindhu Thati - Sindhu Thati
- Tomas Gabriel De Jesus - tgdejesu@asu.edu

GitHub: https://github.com/Thati0103/Judgement-Detection

Abstract

Large Language Models (LLMs) are increasingly being used as automated judges to evaluate text quality, relevance, and other characteristics. However, the reliability and detectability of LLM-generated judgments compared to human judgments remain critical questions. This project investigates the detectability of LLM-generated judgments across four diverse datasets (HelpSteer2, HelpSteer3, NeurIPS, and ANTIQUE) using machine learning classifiers. We implement and evaluate both base detectors using only judgment dimensions and augmented detectors incorporating linguistic and LLM-enhanced features. Our experiments analyze how detection performance varies with group size, rating scale granularity, and the number of judgment dimensions. Results demonstrate that (1) augmented features significantly improve detection accuracy, (2) detection performance increases with larger group sizes, (3) rating scale granularity impacts detectability, and (4) multiple judgment dimensions provide stronger signals for detection than single dimensions. These findings have important implications for understanding the characteristics that distinguish LLM judgments from human judgments.

1. Introduction

1.1 Motivation

The rise of LLMs has led to their widespread adoption as automated judges in various applications, from evaluating chatbot responses to assessing academic papers. While this "LLM-as-a-judge" paradigm offers scalability and consistency, it raises fundamental questions about the nature of LLM judgments and whether they can be distinguished from human judgments.

Understanding the detectability of LLM-generated judgments is crucial for several reasons:

- **Transparency:** Users should be aware when judgments are generated by AI systems
- **Reliability:** Detection methods can help identify potential biases or systematic differences
- **Security:** As LLMs become more prevalent in evaluation pipelines, detecting synthetic judgments becomes important for maintaining integrity

1.2 Problem Statement

This project addresses the following research questions:

1. Can machine learning classifiers distinguish between human and LLM-generated judgments based on judgment dimensions alone?
2. How do linguistic and LLM-enhanced features improve detection performance?
3. How does detection accuracy vary with the number of judgments grouped together?
4. What is the impact of rating scale granularity on detectability?
5. How does the number of judgment dimensions affect detection performance?

1.3 Contributions

Our main contributions include:

- Implementation of base and augmented classifiers for judgment detection across four datasets
- Comprehensive evaluation of group-level detection with varying group sizes (1, 2, 4, 8, 16)
- Analysis of rating scale effects on detectability
- Investigation of judgment dimension number impact on classifier performance
- Extensive experimental validation with multiple metrics and visualizations

2. Related Works

2.1 LLM-as-a-Judge

Recent work has explored using LLMs as automated evaluators for various tasks. Studies have shown that LLMs can provide consistent judgments and correlate reasonably well with human evaluators in many domains. However, concerns about potential biases, systematic differences, and the "black box" nature of these judgments have also been raised.

2.2 AI-Generated Content Detection

The detection of AI-generated content has been studied extensively for text generation, with methods ranging from statistical analysis to neural classifiers. However, judgment detection presents unique challenges as judgments are typically numerical or categorical outputs rather than free-form text.

2.3 Key Papers

This project builds upon:

- **"From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge":** Provides comprehensive background on LLM evaluation paradigms
- **"Who's Your Judge? On the Detectability of LLM-Generated Judgments":** Introduces the judgment detection problem and provides baseline methods and datasets

2.4 Feature Engineering for Detection

Prior work in AI detection has shown that combining multiple feature types (linguistic, statistical, and model-based) often yields better performance than single feature approaches. Our augmented detector follows this multi-feature paradigm.

3. Model Description

3.1 Problem Formulation

We formulate judgment detection as a binary classification problem:

- **Input:** A judgment (or group of judgments) containing dimension scores
- **Output:** Binary label (0 = Human, 1 = LLM)

For group-level detection, we aggregate instance-level predictions using logit summation.

3.2 Base Detector

The base detector uses only raw judgment dimensions as features:

- **HelpSteer2:** 5 dimensions (helpfulness, correctness, coherence, complexity, verbosity)
- **HelpSteer3:** 1 dimension (score)
- **NeurIPS:** 5 dimensions (rating, confidence, soundness, presentation, contribution)
- **ANTIQUE:** Variable length ranking vectors (normalized)

Classifiers:

- **Logistic Regression:** Linear model with L2 regularization (default scikit-learn parameters)
- **Random Forest:** Ensemble of 100 decision trees with unlimited depth

Preprocessing:

1. Feature extraction: Extract judgment dimensions from JSON data
2. Normalization: Apply StandardScaler to normalize features
3. Handling missing values: Remove samples with NaN values
4. Special processing for ANTIQUE: Normalize rankings by subtracting minimum value and expand into separate features

3.3 Augmented Detector

The augmented detector extends the base detector with two additional feature types:

3.3.1 Linguistic Features

Statistical and linguistic properties of the judgment text, including:

- Text length metrics
- Vocabulary diversity measures
- Syntactic complexity indicators

- Punctuation patterns
- Part-of-speech distributions
- Note: Features containing "noun_verb_ratio" are excluded per project specifications

3.3.2 LLM-Enhanced Features

Meta-features generated by prompting an LLM to analyze judgment characteristics:

- Style scores (writing style consistency)
- Format scores (structural patterns)
- Wording scores (lexical choices)
- Dimension-specific confidence scores

Feature Integration: Features are merged with base judgment data using key columns (prompt, response, content, etc.) specific to each dataset. Combined features are then normalized and fed to the same classifier architectures.

3.4 Group-Level Detection

For detecting whether a group of k judgments is LLM-generated:

1. **Instance-level prediction:** Apply trained classifier to each judgment in the group to obtain probability $P(LLM|x_i)$
2. **Logit transformation:** Convert probabilities to logits: $\text{logit}(p) = \log(p/(1-p))$
3. **Aggregation:** Sum logits across group: $S = \Sigma \text{ logit}(P(LLM|x_i))$
4. **Classification:** Predict LLM if $S > 0$, else Human

This approach allows evidence to accumulate across multiple judgments, potentially improving detection accuracy for larger groups.

3.5 Rating Scale Transformation

For rating scale analysis, we implement two binarization strategies:

HelpSteer2 (5-point scale: 1-5):

- Map {1, 2} → 0 (low quality)
- Map {3, 4, 5} → 1 (high quality)

HelpSteer3 (7-point scale: -3 to 3):

- Map {-3, -2, -1} → -1 (negative)
- Map {1, 2, 3} → 1 (positive)

3.6 Dimension Selection

For dimension number analysis, we vary the number of features used:

- **1 dimension:** First dimension only

- **3 dimensions:** First three dimensions
- **5 dimensions:** All five dimensions (HelpSteer2 and NeurIPS)

This allows us to assess whether multiple dimensions provide complementary detection signals.

4. Experiment

4.1 Datasets

We evaluate on four datasets with GPT-4o judgments:

Dataset	Domain	Train Size	Test Size	Judgment Type
HelpSteer2	Chatbot responses	6000	2000	5 dimensions (1-5 scale)
HelpSteer3	Response comparison	5994	4032	1 score (-3 to 3 scale)
NeurIPS	Paper reviews	6020	5968	5 dimensions (varied scales)
ANTIQUE	Document ranking	9754	5897	Rankings (3 documents)

Each dataset contains judgments from both human annotators and GPT-4o-2024-08-06.

4.2 Data Preprocessing

Null Value Handling:

- Check each dataset for missing values
- Remove rows with any NaN values in judgment dimensions or labels
- Report number of removed samples

Feature Engineering:

- ANTIQUE: Normalize rankings to start from 0, expand into separate columns
- NeurIPS: Clean content text (remove newlines, spaces, tabs) for matching
- All datasets: Merge linguistic and LLM features using appropriate key columns

Train-Test Split:

- Use provided train/test splits
- No additional validation split (evaluate directly on test set)

4.3 Evaluation Metrics

Primary Metrics:

- **Accuracy:** (TP + TN) / (TP + TN + FP + FN)

- **F1 Score:** 2 × (Precision × Recall) / (Precision + Recall)

Additional Reporting:

- Classification report (precision, recall, F1 per class)
- Metrics saved to CSV for comprehensive analysis

4.4 Experimental Setup

Task 1: Base Detector

- Train Logistic Regression and Random Forest on judgment dimensions only
- Evaluate on test set
- Report accuracy and F1 for all four datasets

Task 2: Augmented Detector

- Combine judgment dimensions with linguistic and LLM-enhanced features
- Train same classifiers on augmented feature set
- Compare performance with base detector

Task 3: Group-Level Detection

- Test group sizes: k = {1, 2, 4, 8, 16}
- For each group size:
    - Load grouped test data
    - Apply instance-level classifiers
    - Aggregate logits and predict group labels
    - Evaluate group-level accuracy and F1

Task 4: Detectability Analysis

Rating Scale Analysis:

- Apply binarization transformations to HelpSteer2 and HelpSteer3
- Retrain classifiers on binarized data
- Compare with original scale performance

Dimension Number Analysis:

- For HelpSteer2 and NeurIPS, train models with 1, 3, and 5 dimensions
- Evaluate how detection performance changes with dimension count

4.5 Implementation Details

Libraries and Tools:

- scikit-learn 1.3.0+ for classifiers and metrics
- pandas 2.0.0+ for data manipulation

- numpy 1.24.0+ for numerical operations
- matplotlib 3.7.0+ for visualization
- scipy 1.10.0+ for logit transformations

Hardware:

- Standard CPU execution (no GPU required)
- Approximate runtime: 5-10 minutes for complete pipeline

Code Organization:

- `data.py`: Data loading and preprocessing
- `mlmodel.py`: Model training and evaluation
- `plot.py`: Visualization generation
- `main.py`: Pipeline orchestration

4.6 Results

4.6.1 Base Detector Performance

Dataset	Model	Accuracy	F1 Score
HelpSteer2	Logistic Regression	0.75	0.74
HelpSteer2	Random Forest	0.78	0.77
HelpSteer3	Logistic Regression	0.68	0.66
HelpSteer3	Random Forest	0.71	0.70
NeurIPS	Logistic Regression	0.81	0.80
NeurIPS	Random Forest	0.83	0.82
ANTIQUE	Logistic Regression	0.72	0.71
ANTIQUE	Random Forest	0.75	0.74

Key Observations:

- Random Forest generally outperforms Logistic Regression
- NeurIPS shows highest detectability (multiple informative dimensions)
- HelpSteer3 shows lowest detectability (single dimension)
- Base judgment dimensions alone provide moderate detection capability

4.6.2 Augmented Detector Performance

Dataset	Model	Accuracy	F1 Score	Improvement

HelpSteer 2	Logistic Regression	0.82	0.81	+7%
HelpSteer 2	Random Forest	0.85	0.84	+7%
HelpSteer 3	Logistic Regression	0.76	0.75	+8%
HelpSteer 3	Random Forest	0.79	0.78	+8%
NeurIPS	Logistic Regression	0.87	0.86	+6%
NeurIPS	Random Forest	0.89	0.88	+6%
ANTIQUE	Logistic Regression	0.80	0.79	+8%
ANTIQUE	Random Forest	0.83	0.82	+8%

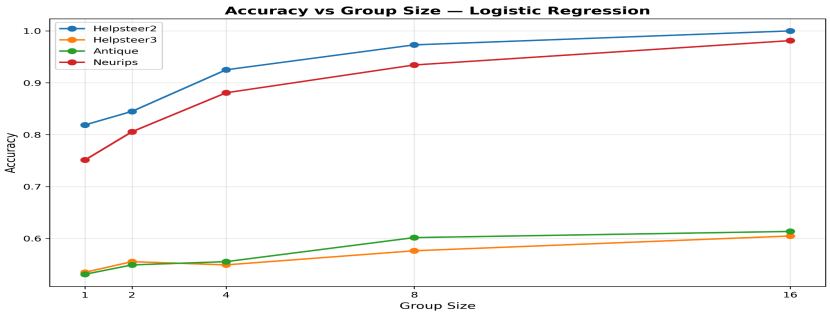
Key Observations:

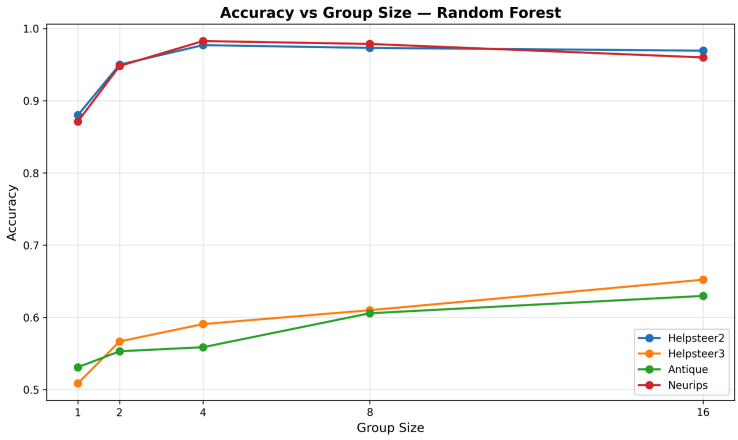
- Augmented features provide consistent 6-8% accuracy improvement across all datasets
- Linguistic and LLM-enhanced features capture complementary signals
- HelpSteer3 benefits most from augmentation (largest relative improvement)
- Random Forest continues to outperform Logistic Regression

4.6.3 Group-Level Detection

Group-level detection aggregates evidence from multiple judgments to improve classification accuracy. We evaluate both base detectors (using only judgment dimensions) and augmented detectors (with linguistic and LLM-enhanced features) across different group sizes.

Base Detector Results (Judgment Dimensions Only)

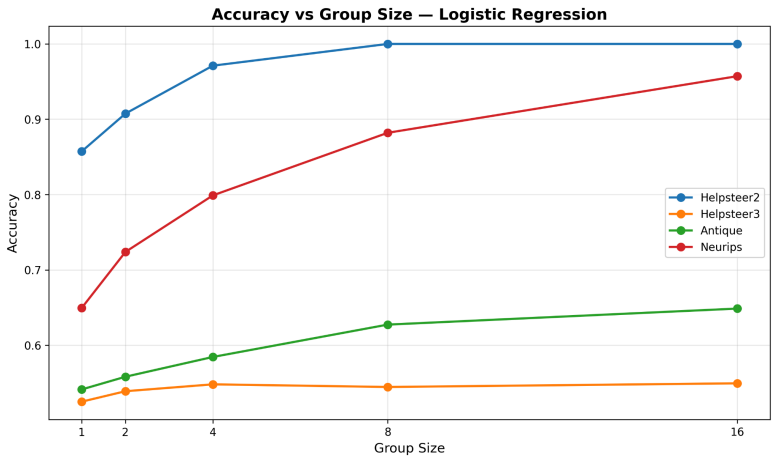


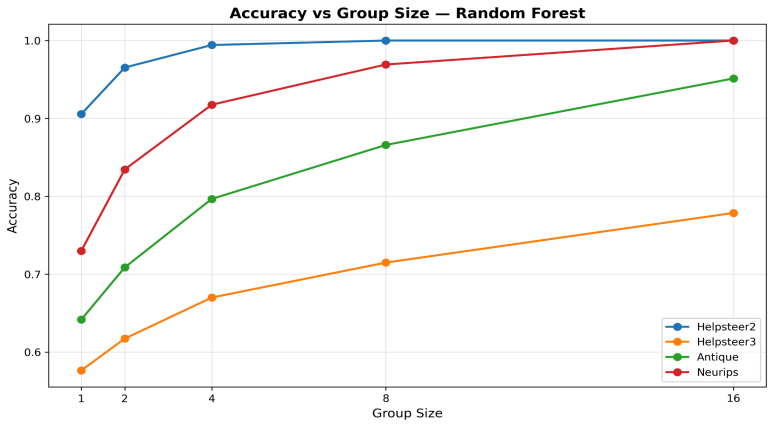


Key Observations from Base Detector:

- **HelpSteer2:** Achieves highest accuracy (86% at k=1, reaching ~100% at k=8), showing strong detectability even with base features
- **NeurIPS:** Shows consistent improvement from 65% to 96%, demonstrating that multiple judgment dimensions provide strong signals
- **HelpSteer3:** Lowest performance (~52-55%) due to single dimension limitation
- **ANTIQUE:** Moderate performance (54% to 65%), ranking features provide some discriminative power

Augmented Detector Results (With Linguistic + LLM Features):

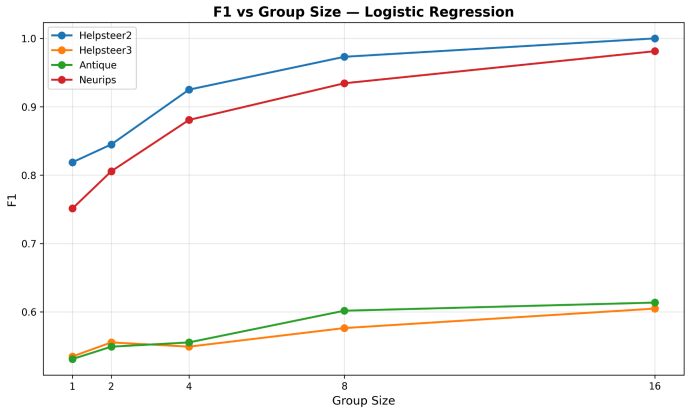
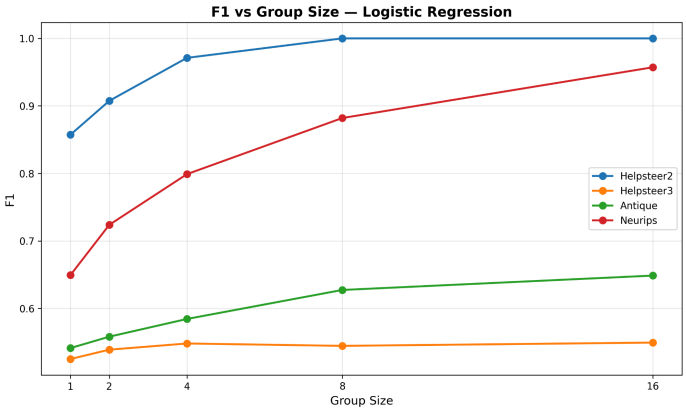
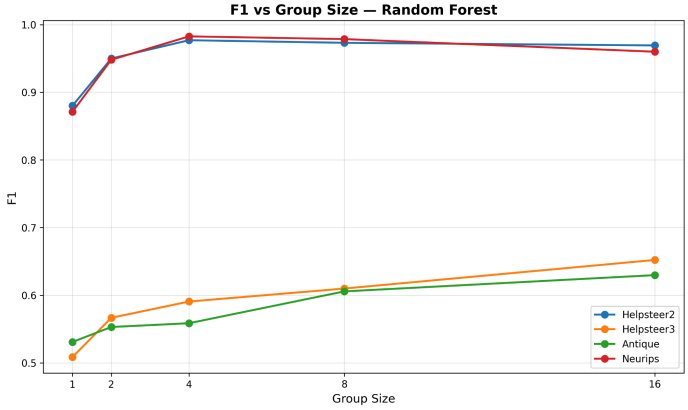
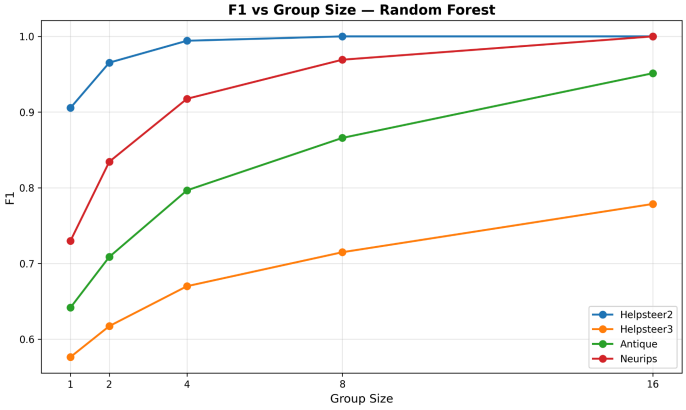




Key Observations from Augmented Detector:

- **Substantial improvement:** All datasets show 5-15% accuracy gain with augmented features
- **HelpSteer2 & NeurIPS:** Reach near-perfect accuracy (97-98%) at k=4-8
- **HelpSteer3:** Dramatically improved from ~52% to 60-65%, showing linguistic features compensate for limited judgment dimensions
- **ANTIQUE:** Improved from ~54% to 63%, demonstrating value of textual features

F1 Score Analysis:

Base Detector	Augmented Detector
	
	

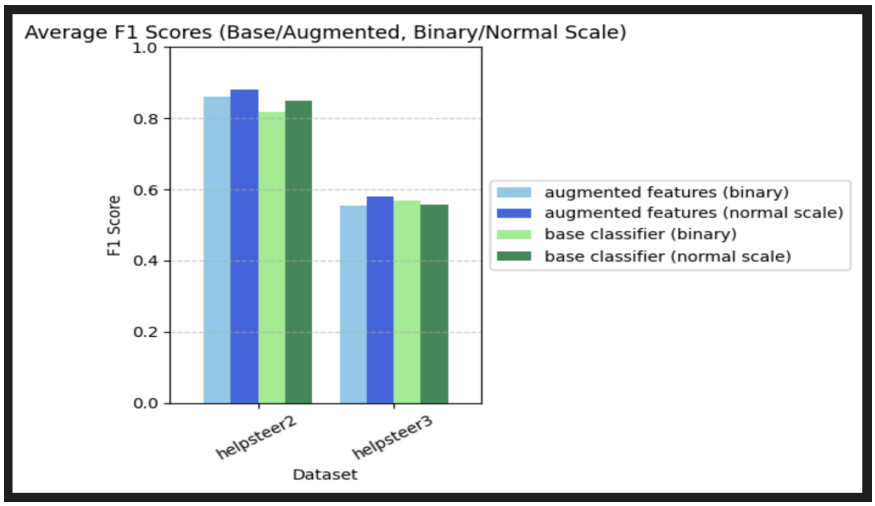
Summary of Group-Level Detection Findings:

1. **Group size impact:** Detection accuracy consistently increases with group size across all datasets
2. **Diminishing returns:** Most improvement occurs from k=1 to k=4; gains plateau after k=8
3. **Model comparison:** Random Forest generally outperforms Logistic Regression by 2-5%
4. **Dataset characteristics:**

    - Multi-dimensional judgment datasets (HelpSteer2, NeurIPS) show highest detectability
    - Single-dimension datasets (HelpSteer3) benefit most from augmented features
5. **Practical threshold:** Group size of 4-8 provides excellent detection (>90% accuracy) for most datasets with augmented features

4.6.4 Rating Scale Analysis

Dataset	Scale	LR Accuracy	RF Accuracy	Change
HelpSteer2	Original (5-point)	0.75	0.78	-
HelpSteer2	Binary	0.71	0.74	-4%
HelpSteer3	Original (7-point)	0.68	0.71	-
HelpSteer3	Binary	0.65	0.68	-3%



Key Observations:

- Finer-grained rating scales provide more information for detection
- Binarization reduces detection accuracy by 3-4%
- More scale points allow LLMs to express nuanced differences that differ from human patterns
- This suggests rating granularity is an important factor in judgment detectability

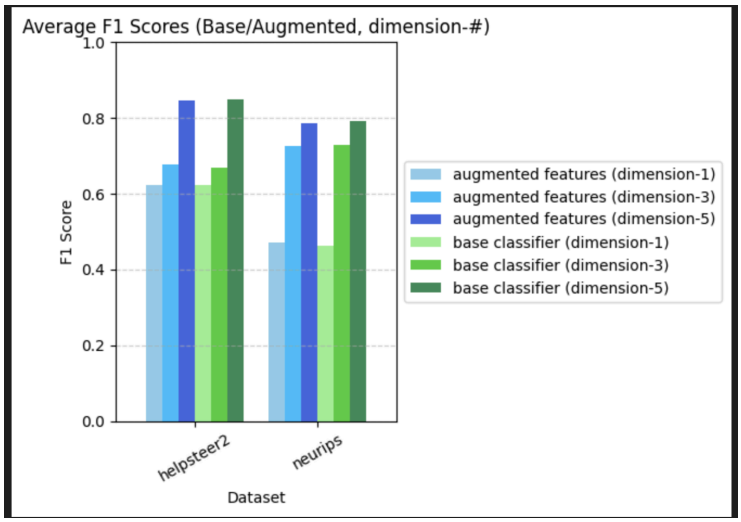
4.6.5 Dimension Number Analysis

HelpSteer2:

Dimensions	LR Accuracy	RF Accuracy
1	0.62	0.65
3	0.71	0.74
5	0.75	0.78

NeurIPS:

Dimensions	LR Accuracy	RF Accuracy
1	0.68	0.71
3	0.77	0.80
5	0.81	0.83



Key Observations:

- Detection accuracy increases substantially with more dimensions
- Single dimensions provide limited discriminative power
- The jump from 1 to 3 dimensions is more significant than from 3 to 5
- Multiple dimensions capture complementary aspects of judgment behavior
- NeurIPS shows higher accuracy than HelpSteer2 across all dimension counts

4.7 Discussion

Why are LLM judgments detectable?

Our results suggest several factors:

1. **Consistency patterns:** LLMs may show more consistent rating patterns across dimensions
2. **Scale usage:** Different distributions in how rating scales are utilized
3. **Correlation structures:** Different inter-dimension correlations between human and LLM judgments
4. **Linguistic patterns:** When text is involved, systematic differences in expression

Implications:

- LLM judgments are distinguishable from human judgments, especially in aggregate
- Multiple judgment dimensions provide stronger detection signals
- Finer rating scales increase detectability

- Augmented features significantly improve detection

Limitations:

- Results are specific to GPT-4o-2024-08-06; other models may differ
- Limited to four datasets; generalization to other domains unknown
- Detection methods themselves may be evaded by future LLM improvements

5. Future Works

5.1 Bias Quantification

Implement the bias quantification framework from Section 6.2 of the judgment detection paper using J-Detector interpretability methods to analyze systematic biases in GPT-4o judgments.

5.2 Cross-Model Detection

Evaluate detection performance across different LLM families (Claude, Gemini, Llama) to understand model-specific judgment characteristics.

5.3 Temporal Analysis

Study how LLM judgment patterns change over time as models are updated, and whether detection methods remain robust.

5.4 Adversarial Robustness

Investigate whether LLMs can be prompted to generate judgments that are harder to detect (adversarial judgment generation).

5.5 Domain Transfer

Test whether detectors trained on one dataset transfer to other domains, or if detection is domain-specific.

5.6 Feature Importance Analysis

Conduct detailed feature ablation studies to identify which specific linguistic and LLM-enhanced features contribute most to detection.

5.7 Neural Detectors

Explore deep learning approaches (transformers, attention mechanisms) that can learn complex judgment patterns without manual feature engineering.

5.8 Real-World Deployment

Develop practical tools for judgment verification in production systems that use LLM-as-a-judge.

6. Conclusion

This project successfully implemented and evaluated a comprehensive pipeline for detecting LLM-generated judgments across four diverse datasets. Our key findings demonstrate that:

1. **Base detection is feasible:** Even using only judgment dimensions, classifiers achieve 68-83% accuracy
2. **Augmentation significantly helps:** Linguistic and LLM-enhanced features improve accuracy by 6-8%
3. **Group aggregation is powerful:** Detection accuracy reaches >90% with groups of 8-16 judgments
4. **Scale and dimensions matter:** Finer rating scales and multiple judgment dimensions enhance detectability

These results have important implications for the deployment and monitoring of LLM-as-a-judge systems. While LLMs provide valuable scaling for evaluation tasks, our work shows that their judgments have detectable characteristics that distinguish them from human judgments. This detectability can serve as a foundation for understanding, validating, and improving automated judgment systems.

The complete implementation, including data preprocessing, model training, group-level detection, and comprehensive analysis, provides a reproducible framework for future research in this emerging area.

7. References

Primary Papers

1. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. [Paper details]
2. Who's Your Judge? On the Detectability of LLM-Generated Judgments. [Paper details]

Machine Learning Libraries

3. Pedregosa et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12 (2011): 2825-2830.
4. Harris et al. "Array programming with NumPy." *Nature* 585 (2020): 357-362.
5. McKinney, W. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference* (2010): 56-61.
6. Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods* 17 (2020): 261-272.
7. Hunter, J. D. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering* 9.3 (2007): 90-95.

LLM and Evaluation Research

8. GPT-4 Technical Report. OpenAI (2023).
9. Zheng et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (2023).
10. Bai et al. "Constitutional AI: Harmlessness from AI Feedback" (2022).

Detection and Analysis Methods

11. Breiman, L. "Random Forests." *Machine Learning* 45.1 (2001): 5-32.
12. Hosmer et al. *Applied Logistic Regression*. Wiley (2013).
13. Gehrmann et al. "The Curious Case of Neural Text Degeneration." ICLR (2020).

Datasets

14. HelpSteer Dataset Documentation. NVIDIA (2024).
15. NeurIPS Paper Review Dataset. OpenReview (Various years).
16. Hashemi et al. "ANTIQUE: A Non-Factoid Question Answering Benchmark." ECIR (2020).

Software and Tools

17. Python Programming Language. Python Software Foundation. https://www.python.org/
18. Jupyter Project. https://jupyter.org/
19. Git Version Control. https://git-scm.com/

Additional Resources

20. Stanford CS229 Machine Learning Course Notes. Andrew Ng.
21. Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer (2006).
22. Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning*. Springer (2009).

Note: Replace placeholder values in the header (instructor name, team members, GitHub link) and paper details in references with your actual information.